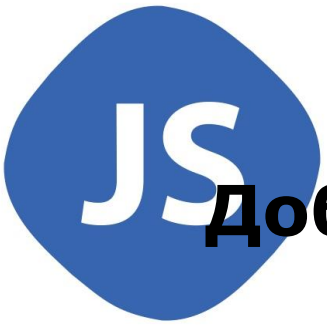


Добавление и удаление узлов

- Создание узлов
- Вставка узлов
- Замена и клонирование узлов
- Удаление узлов
- Живая и статическая коллекции

Создание узлов

Создание текстовых узлов

Создание **текстового узла** в JavaScript осуществляется посредством метода **createTextNode**.

Синтаксис

```
node = document.createTextNode('text');
```

В аргументе createTextNode необходимо поместить текст, который должен иметь этот текстовый узел.

Например, создадим текстовый узел с текстом "Новый текстовый узел":

```
let txt = document.createTextNode('Новый текстовый узел');
```

Созданный таким образом текстовый узел **не будет являться** частью DOM-дерева, а значит, не будет содержаться в HTML-документе.

Для **вставки узлов в HTML-страницу** необходимо воспользоваться специальным методом.

Создание узлов-элементов



Создание **узла-элемента** в JavaScript выполняется с помощью метода **createElement**.

Синтаксис

```
// node – переменная, в которую сохраним созданный элемент
node = document.createElement('tag');
```

Вместо **tag** необходимо указать тег того элемента, который нужно создать.

Например, создадим элемент div:

```
let div = document.createElement('div');
```

При создании нового элемента ему сразу можно назначить некоторый **набор свойств**:

```
// создаем узел-элемент div
var div = document.createElement('div');
div.className = 'myclass';
div.id = 'myid';
div.innerHTML = 'Привет, мир!';
```

example_1. Создание узла-элемента

```
<script>
      var div = document.createElement('div');
      div.className = 'myclass';
      div.id = 'myid';
      div.innerHTML = '<h1>Партия - ум, честь и совесть нашей
      эпохи!</h1>';
      // ищем div в документе
      console.log (document.body); // нет его... :(
</script>
```



Созданный элемент пока еще не является частью DOM-дерева, а значит, не **содержится в HTML-документе**. Если мы сейчас обновим страницу, то ничего на ней не увидим.

Для **вставки узлов-элементов в HTML-страницу** существуют специальные методы.

Вставка узлов

Чтобы DOM-узел был показан на странице, его необходимо вставить в **целевой узел** документа.

В JavaScript **методы вставки узлов** условно разделяют следующим образом:

1. **Старые методы**
   - **appendChild**;
   - **insertBefore**.
2. **Современные методы**
   - node.**append** – для добавления узлов или строк в конец node;
   - node.**prepend** – для вставки узлов или строк в начало node;
   - node.**before** – для вставки узлов или строк до node;
   - node.**after** – для вставки узлов или строк после node.

Пример действия методов:

```
<!-- before -->
<ul>
    <!-- prepend -->
    <li>PHP</li>
    <li>MySQL</li>
    <!-- append -->
</ul>
```

```
<!-- after -->
```

3. Набор методов InsertAdjacent

- node.**insertAdjacentElement**();
- node.**insertAdjacentHTML**();
- node.**insertAdjacentText**().

Старые методы рассматривать не будем. А вот современные методы вставки и методы InsertAdjacent рассмотрим достаточно подробно.

Набор современных методов

example_2. Вставка текстового узла

```
<script>
	let txt = document.createTextNode('Догоним и перегоним Америку!');
	// вставляем текстовый узел в конец документа
	document.body.append(txt);
	// проверим наличие текстового узла
	console.log (document.body);  // есть такой ;)
</script>
```

example_3. Вставка узла-элемента

```
<script>
	var div = document.createElement('div');
	div.className = 'myclass';
	div.id = 'myid';
	div.innerHTML = '<h1>Даёшь стране угля!</h1>';
	// выбираем целевой элемент
	let target = document.getElementsByTagName("header")[0];
```

```
    // вставляем узел-элемент в документ
    target.after(div);
    // ищем div в документе
    console.log (document.body); // находим ;)
</script>
```

Обратите внимание, методы могут вставлять не только объекты-теги, но и **текстовые элементы** как строки.

Рассмотрим вставку строковых значений на примере метода append.

example_4. Вставка строкового значения

```
<script>
    // вставляем текстовый (!) узел в документ
    document.body.append("<h2>Сегодня он играет джаз, а завтра Родину продаст</h2>");
    // проверим наличие текстового узла
    console.log (document.body);
</script>
```

Рассмотрим вставку узла-элемента и текстового узла в **классическом примере вставки объекта** на HTML-страницу.

example_5. Вставка узлов

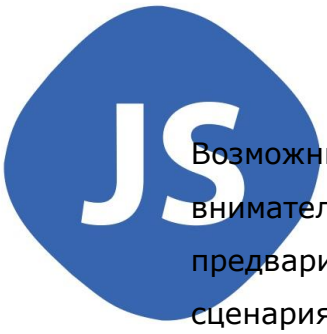
```
<script>
    // создаем узел-элемент тега <p>
    let node_p = document.createElement("p");
    // создаем текстовый узел
    let node_txt = document.createTextNode('Жизнь прекрасна, невзирая на недочеты.');
    // в узел node_p вставляем текстовый узел node_txt
```

```
        node_p.append(node_txt);
        // выбираем элемент <div id="citate">
        let div = document.querySelector("div#citate");
        // в узел-элемент div вставляем узел-элемент node_p
        div.prepend(node_p);
</script>
```

В следующем примере созданному **узлу-элементу** перед вставкой в документ назначим атрибуты **class** и **id** для визуального выделения на странице.

example_6. Работа с узлами DOM

```
<script>
        // переменная с текстом для текстового узла
        let txt = 'Автомобиль — не роскошь, а средство передвижения.';
        // выбираем элемент <div id="citate">
        let div = document.querySelector("div#citate");
        // создаем узлы
        let node_p = document.createElement("p");
        let node_txt = document.createTextNode(txt);
        // текстовый узел вставляем в узел-элемент p
        node_p.append(node_txt);
        // назначаем узлу свойства
        node_p.className = "sel";
        node_p.id = "par";
        // вставляем новый узел в элемент div
        div.append(node_p);
        // выведем узел в консоль
        console.log(node_p);
</script>
```



Возможны более **сложные манипуляции элементами** на странице. Будьте внимательны при создании и вставки узлов. Можете использовать предварительный **вывод в консоль** узлов, для отслеживания хода выполнения сценария.

example_7. Сложные манипуляции с узлами-элементами

```
<script>
      // создаем элемент маркированного списка
      let ul = document.createElement("ul");
      // создаеми пункты маркированного списка
      // создаем текстовые узлы для пунктов списка
      let li1 = document.createElement("li");
      let txt1 = document.createTextNode("режиссёр: Алла Сурикова");
      li1.append(txt1);
      let li2 = document.createElement("li");
      let txt2 = document.createTextNode("год: 1987");
      li2.append(txt2);
      let li3 = document.createElement("li");
      let txt3 = document.createTextNode("страна: СССР");
      li3.append(txt3);
      // ----------
      // вставляем пункты в маркированный список
      ul.append(li1, li2, li3);
      // маркированный список выводим в консоль
      console.log(ul);
      // ----------
      // создадим пункт нумерованного списка
      let li = document.createElement("li");
      let txt = document.createTextNode("Человек с бульвара
```



```
Капуцинов:");
      li.append(txt);
      // ----------
      // в пункт нумерованного списка добавим созданный элемент <ul>
      li.append(ul);
      // ----------
      // выберем элемент списка <ol>
      let ol = document.querySelector("ol");
      // теперь всю созданную ветвь добавим в элемент списка <ol>
      ol.append(li);
</script>
```

Достаточно трудоемкий процесс, особенно если речь идет о сложной структуре представления данных.

Пример из листинга example_7 можно переписать в **упрощенном варианте** исполнения. При этом нужно помнить, **такой способ будет работать медленнее**.

example_8. Так создавать узле гораздо менее хлопотно

```
<script>
      // создаем элемент списка
      let li = document.createElement("li");
      // менее трудоемкий но и менее быстрый способ вставки
      li.innerHTML = `
      Человек с бульвара Капуцинов:
            <ul>
                  <li>режиссёр: Алла Сурикова</li>
                  <li>год: 1987</li>
                  <li>страна: СССР</li>
```

```
            </ul>
        `;
        // выберем целевой список <ol>
        let ol = document.querySelector("ol");
        // добавим элемент <li> в целевой список <ol>
        ol.prepend(li);
</script>
```

Протестируем методы **before** и **after**.

example_9. Методы для вставки before и after

```
<script>
        // выбор целевого элемента (нумерованного списка)
        let target = document.querySelector("ol");
        // создаем первый узел h2
        let h1 = document.createElement("h2");
        let txt1 = document.createTextNode("Джентльмены удачи");
        h1.style.color = "red";
        h1.append(txt1);
        console.log(h1)
        // создаем второй узел h2
        let h2 = document.createElement("h2");
        h2.style.color = "blue";
        h2.append("Гараж");
        console.log(h2);
        // вставим первый узел перед списком
        target.before(h1);
        // вставим второй узел после списка
        target.after(h2);
```

```
</script>
```

Набор методов InsertAdjacent

В JavaScript имеется набор методов **insertAdjacent**, которые позволяют вставить один или несколько узлов в указанную **позицию** (position) относительно **выбранного узла** (node).

Всего существует 3 таких метода.

Синтаксис

```
// 1. вставка элемента (element)
node.insertAdjacentElement(position, element);
// 2. вставка строки HTML (htmlString)
node.insertAdjacentHTML(position, htmlString);
// 3. вставка текстовой строки (string)
node.insertAdjacentText(position, string);
```

Значение **position**, может быть, одним из следующих:

- node.**beforebegin** – непосредственно перед node;
- node.**afterbegin** – перед первым дочерним узлом node;
- node.**beforeend** – после последнего дочернего узла node;
- node.**afterend** – сразу после node.

```
<!-- beforebegin -->
<ul>
    <!-- afterbegin -->
    <li>HTML</li>
    <li>CSS</li>
    <!-- beforeend -->
</ul>
<!-- afterend -->
```

Пример использования метода **insertAdjacentElement**.

example_10. Метод insertAdjacentElement

```
<script>
	// выберем целевой элемент
	let ul = document.querySelector("ul");
	// создадим 1-й элемент
	let elem1 = document.createElement("h2");
	elem1.append("Технологии для Backend-разработчика");
	console.log(elem1);
	// создадим 2-й элемент
	let elem2 = document.createElement("li");
	elem2.append("<b>Laravel</b>");  // append не понимает теги
	// создадим 3-й элемент
	let elem3 = document.createElement("li");
	elem3.innerText = "<b>Python</b>";  // innerText не понимает теги
	// создадим 4-й элемент
	let elem4 = document.createElement("div");
	elem4.innerHTML = "<h3>На странице группы pechora-PROger в
	VKонтакте</h3>";
	// выполняем вставку
	ul.insertAdjacentElement('beforebegin', elem1);
	ul.insertAdjacentElement('afterbegin', elem2);
	ul.insertAdjacentElement('beforeend', elem3);
	ul.insertAdjacentElement('afterend', elem4);
</script>
```

Пример использования метода **insertAdjacentHTML**.

example_11. Метод insertAdjacentHTML

```
<script>
	// выберем целевой элемент
	let ul = document.querySelector("ul");
	// выполняем вставку
	ul.insertAdjacentHTML('beforebegin', "<h2>Технологии для Frontend-разработчика</h2>");
	ul.insertAdjacentHTML('afterbegin', "<li>JavaScript</li>");
	ul.insertAdjacentHTML('beforeend', "<li>Bootstrap</li>");
	ul.insertAdjacentHTML('afterend', "<h3>На странице группы pechora-PROger в VKонтакте</h3>");
	// посмотрите консоль разработчика вкладку Элементы обязательно
</script>
```

Пример использования метода **insertAdjacentText**.

example_12. Метод insertAdjacentText

```
<script>
	// выберем целевой элемент
	let ul = document.querySelector("ul");
	ul.insertAdjacentText('beforebegin', "Инструментарий графического дизайнера");
	ul.insertAdjacentText('afterbegin', "Adobe Photoshop");
	// обратите внимание теги в текстовом формате
	ul.insertAdjacentText('beforeend', "<li>CorelDRAW Graphics Suite</li>");
	ul.insertAdjacentText('afterend', "<h3>На странице группы pechora-PROger в VKонтакте</h3>");
</script>
```

Примечание. Сравните вставку контента через свойство **innerHTML** и через методы **insertAdjacent**. В первом случае вы просто **замещаете** контент выбранного узла, во втором - имеете возможность выбора **позиции вставки**. При внешней схожести, набор методов insertAdjacent имеет более гибкий механизм взаимодействия с DOM документа.

Замена и клонирование узлов

Замена узлов

Замену одного узла другим (другими) в JavaScript можно выполнить с помощью следующих методов:

- **replaceChild** (считается устаревшим)
- **replaceWith**.

Синтаксис

```
elem.replaceWith(node);

elem.replaceWith(node1, node2);

elem.replaceWith(node1, node2, ..., nodeN);
```

example_13. Метод replaceWith

```
<script>

    // выберем второй элемент исходного списка

    let old_li = document.querySelector("ol li:nth-child(2)");

    // создадим первый элемент списка с текстом

    let new_li1 = document.createElement("li");

    new_li1.innerText = "Белое солнце пустыни";

    // создадим второй элемент списка с текстом

    let new_li2 = document.createElement("li");
```

```
    new_li2.innerText = "Берегись автомобиля";
    // заменим элемент исходного списка на новые элементы
    old_li.replaceWith(new_li1, new_li2);
</script>
```

Клонирование узлов

Для создания копии узла предназначен метод **cloneNode**.

Синтаксис

```
copy = node.cloneNode(deep);
```

где:

- **node** – узел, который нужно клонировать;
- **copy** – переменная, в которую нужно поместить новый узел, который будет копией node;
- **deep** – глубина клонирования (по умолчанию **false**, т.е. выполняется клонирование только самого элемента node без детей); если установить **true**, то node будет скопирован со всеми его детьми.

Например, скопируем нумерованный список **ol** и вставим его в конец элемента **body**.

example_14. Метод cloneNode

```
<script>
    // сделаем клон элемента <ol> и сохраним в переменную
    let ol = document.querySelector("ol").cloneNode(true);
    // выведем в консоль
    console.log(ol);
    // проведем ограничитель
    document.write("<hr>");
    // выведем в браузер клон списка <ol>
```

```
    document.body.append(ol);
</script>
```

Удаление узлов

Удалить узел из DOM можно в JavaScript с помощью методов:

- **removeChild** (считается устаревшим)
- **remove**.

Синтаксис

```
node.remove();
```

example_15. Метод remove

```
<script>
    // выбор второго элемента нумерованного списка
    let li = document.querySelector("ol > li:nth-child(2)");
    // удаление выбранного элемента
    li.remove();
    // удаленный элемент вывели в консоль
    console.log(li);
    document.write("<hr>");
    // вставка в документ удаленного элемента
    document.body.append(li);
</script>
```



Живая и статическая коллекции

Найти несколько DOM-элементов и получить к ним доступ из JavaScript можно разными способами, вне зависимости от способа будет возвращена коллекция DOM-элементов.

> ***К сведению***. DOM-коллекция — это объект (псевдомассив), представляющий список узлов (включающий набор DOM-элементов)

Стоит учесть, что фактически разные способы возвращают разные коллекции:

- **HTMLCollection** — коллекция непосредственно HTML-элементов.
- **NodeList** — коллекция узлов, более абстрактное понятие. Например, в DOM-дереве есть не только узлы-элементы, но также текстовые узлы, узлы-комментарии и другие, поэтому NodeList может содержать другие типы узлов.

При работе с DOM-элементами тип коллекции значительной роли не играет, поэтому для удобства будем рассматривать их как одну сущность — **коллекцию**.

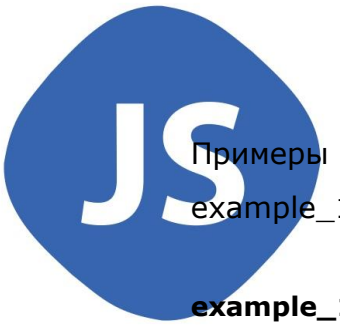
Виды DOM-коллекций:

- **живые** (динамические, реагируют на изменение DOM);
- **статические** (моментальный снимок данных, не реагируют на изменение DOM).

Живая коллекция — коллекция, содержимое которой может изменяться при изменении DOM. Получают методами **getElementsBy***.

Статическая коллекция — коллекция, которая **не изменяется**, если после вызова этого метода на странице появятся новые элементы, подходящие под указанный CSS селектор. Таким образом, если вам необходимо обновить полученную коллекцию после изменения DOM-страницы, то её следует просто сформировать, заново вызвав этот метод с указанным селектором ещё раз. Статическую коллекцию получают методом **querySelectorAll**.





Примеры получения разных видов коллекций приведены в листингах example_16 - example_17.

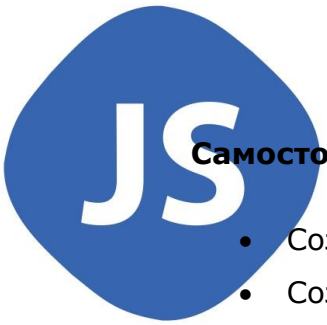
example_16. Создание статичной коллекции

```
<script>
	// querySelectorAll
	// !! статичная коллекция
	var stat = document.querySelectorAll('li');
	console.log("Исходная коллекция:");
	console.log(stat);
	// удаляем элемент из DOM
	stat[0].remove();
	console.log("Коллекция после удаления элемента из DOM:");
	console.log(stat);
</script>
```

example_17. Создание живой коллекции

```
<script>
	// getElementsByTagName
	// !! живая коллекция
	var live = document.getElementsByTagName("li");
	console.log("Исходная коллекция:");
	console.log(live);
	// удаляем элемент из DOM
	live[0].remove();
	console.log("Коллекция после удаления элемента из DOM:");
	console.log(live);
</script>
```





Самостоятельные работы к уроку:

- Создание текстовых узлов.
- Создание узлов-элементов (изображения, ссылки).
- Динамическое создание таблиц.
- Клонирование узлов-элементов.

